

Linear Sorting

Sorting Algorithms

$N =$ # data items to sort

- Comparison Sorts

- Bubble Sort $O(N^2)$

- Heap Sort, Merge Sort $O(N \cdot \log N)$

- Stable vs unstable

9, 1, 3, 1, 4, 4, 3

The diagram illustrates a sequence of numbers: 9, 1, 3, 1, 4, 4, 3. Below the second '1' is an orange '(1)' and below the third '1' is an orange '(2)'. An orange arrow points from the second '1' to the third '1'. Similarly, below the first '4' is an orange '(1)' and below the second '4' is an orange '(2)'. An orange arrow points from the first '4' to the second '4'. This represents comparisons between adjacent elements in a sorting process.

Counting Sort

We take the range R of symbols into consideration.

Input sequence: 8, 8, 9, 0, 1, 3, 9, 0, 3, 5, 3

$R: 0, 1, 2, \dots, 9$

N	index:	0	1	2	3	4	5	6	7	8	9	
	counts:	0 ₂	1 ₁	0	3 ₃	0 ₀	1 ₁	0	0	2 ₂	1 ₂	
R	index:	0	1	2	3	4	5	6	7	8	9	
	endpos:	1 ₀	2 ₁	2	5 ₂	5	6 ₅	6	6	8 ₆	10 ₈	
N	index:	0	1	2	3	4	5	6	7	8	9	10
	output:	0	0	1	3	3	3	5	8	8	9	9

$2 \cdot N + R$

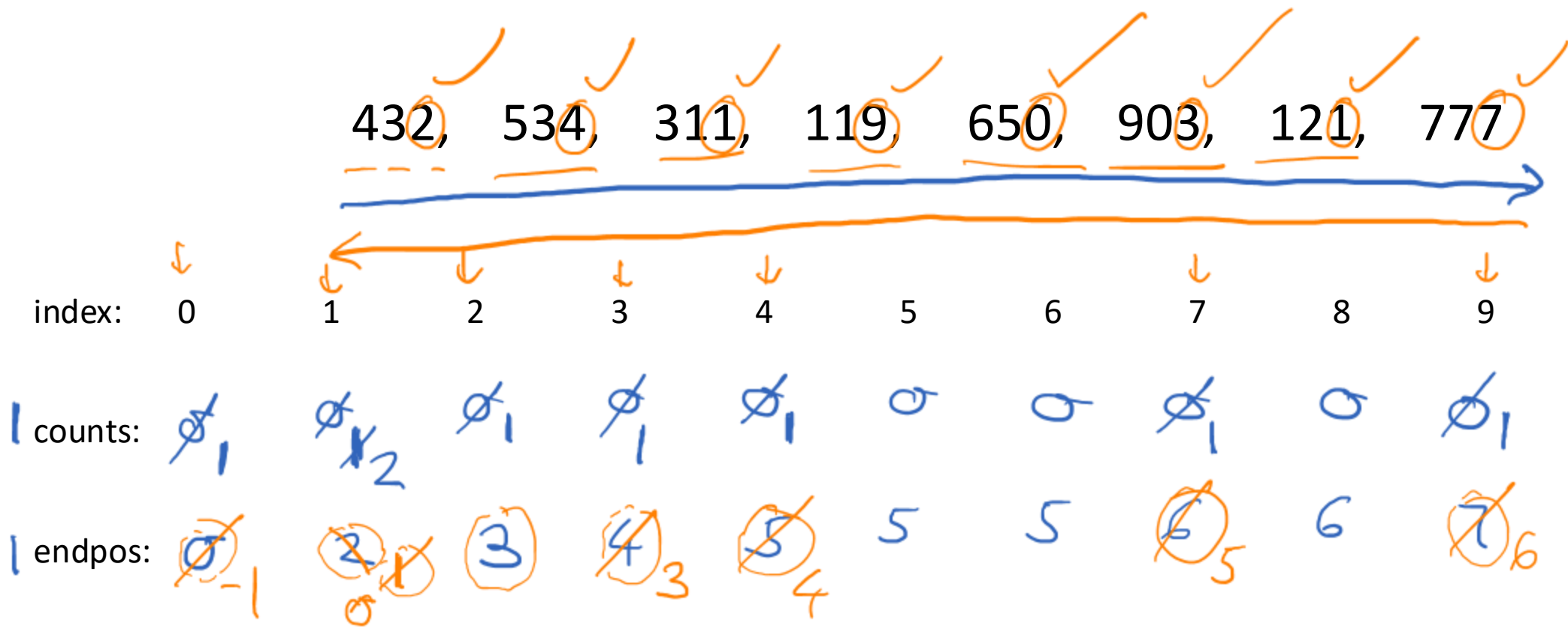
What if the range is big?

432, 534, 311, 119, 650, 903, 121, 777

⇒ Apply Counting Sort
once per positions (3x)

→ Radix Sort

Radix Sort, Iteration 1



output:

0	1	2	3	4	5	6	7
650	311	121	432	903	534	777	119

Radix Sort, Iteration 2

650, 311, 121, 432, 903, 534, 777, 119

index:	0	1	2	3	4	5	6	7	8	9
counts:	1	2	1	2	0	1	0	1	0	0
endpos:	-1	0	2	3	5	5	6	6	7	7

output:	0	1	2	3	4	5	6	7
	903	311	119	121	432	534	650	777

Radix Sort, Iteration 3

903, 311, 119, 121, 432, 534, 650, 777

index:	0	1	2	3	4	5	6	7	8	9
counts:	0	2	0	1	1	1	1	1	0	1
endpos:	-1	-1	1	1	2	3	4	5	6	7

output:	0	1	2	3	4	5	6	7
	119	121	311	432	534	650	777	903



Complexity

N: # of data items to sort

R: range of symbols

Q: length of the data items

Counting Sort: $O(\cancel{2 \cdot N} + \cancel{R})$ const.

const. $O(N)$ const.

Radix Sort: $O(\cancel{Q} \cdot (\cancel{2 \cdot N} + \cancel{R}))$

$O(N)$